

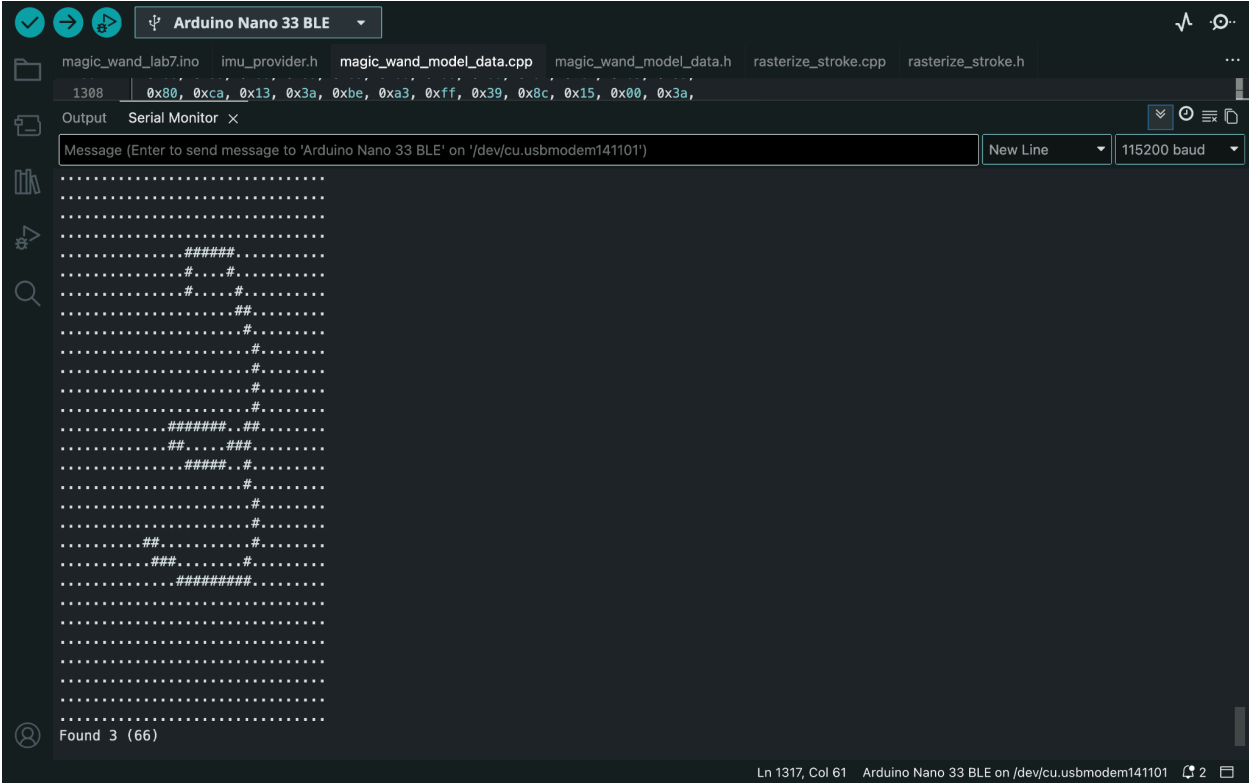
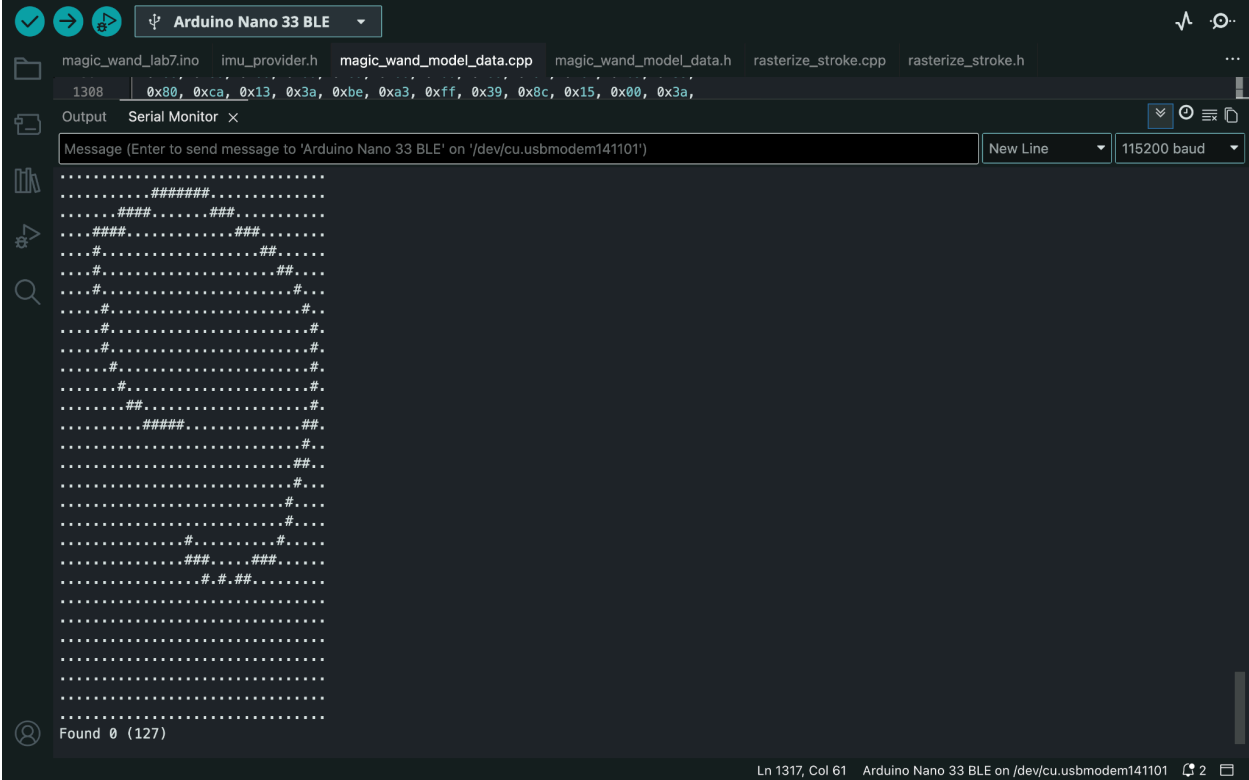
Discussion

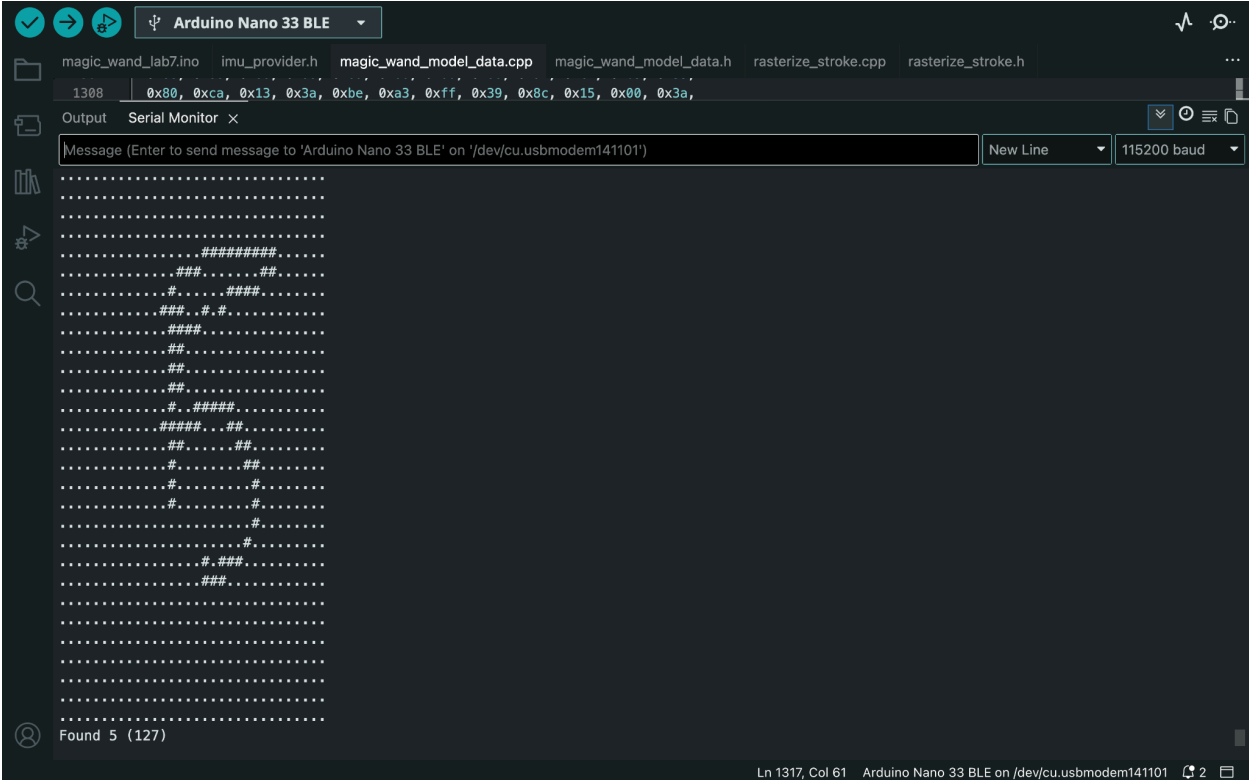
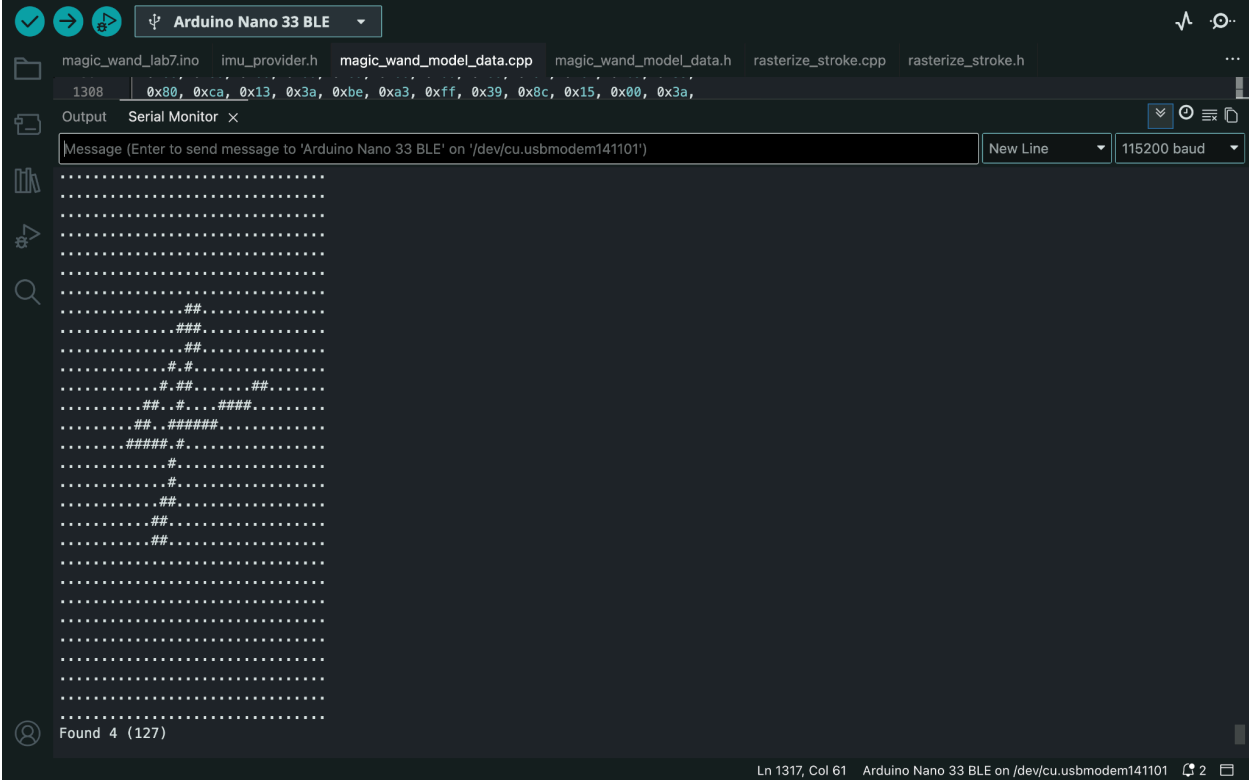
- Between the baseline model and the finetuned model, the baseline model actually performed better than the finetuned in my case. The difference in performance is mainly caused by the poor dataset I collected and used for finetuning.
- For the dataset collection, it was actually split into three person: me, Ben, and Adi. Although we were each supposed to collect 10 labeled numbers from 0-9, the combined dataset ended up with only 74 features, and 19 of those were unlabeled, which means the dataset is really small and skewed.
- Moreover, because the finetuning dataset was collected by three different individuals, that data wasn't tailored to me. Hence during deployment and inference, it wasn't really doing that well for my specific writing habits.
- After identifying this issue, I recollected the data, cleaned the labels, stitched the JSON files together, retrained/finetuned the model, and redeployed it. The new dataset was much more balanced, with 100 total samples and 10 samples per digit class. This led to a clear improvement in the finetuned model. The updated classification report shows an overall accuracy of 93%, with a macro average F1-score of about 0.929. This suggests that the earlier poor result was mainly a data quality problem rather than a limitation of the finetuning method.
- The new model performs especially well on several classes. Digits 1, 2, and 5 all reached perfect precision and recall in the report, meaning the model correctly identified all examples of those digits and did not confuse other digits for them. Digits 3, 4, 7, and 9 also performed strongly, with F1-scores around 0.90–0.95.
- However, the model is still not perfect. The weakest class is digit 8, which has a recall of 0.70 and an F1-score of 0.7778. From the confusion matrix, some 8s were misclassified as 0s and 6s. This makes sense because these digits have visually similar rounded stroke patterns, especially when the wand motion is noisy or inconsistent. There were also smaller confusions such as 0 being classified as 8, 3 as 7, 7 as 0, and 9 as 4. These mistakes show that the model is still sensitive to overlap between similar gesture shapes.
- Overall, the second finetuning attempt was much more successful. The recollected and cleaned dataset improved the model from a poorly generalized finetuned version into one that achieved strong classification performance after redeployment. The main takeaway is that finetuning is highly dependent on dataset quality. A small or mislabeled dataset can make finetuning worse than the baseline, but a clean and balanced dataset can significantly improve the deployed model's performance. Further improvements would likely come from collecting more examples per class, especially for confusing digits like 0, 6, and 8, and collecting more samples under realistic deployment conditions so the model becomes more robust to variation in writing speed, size, and motion style.

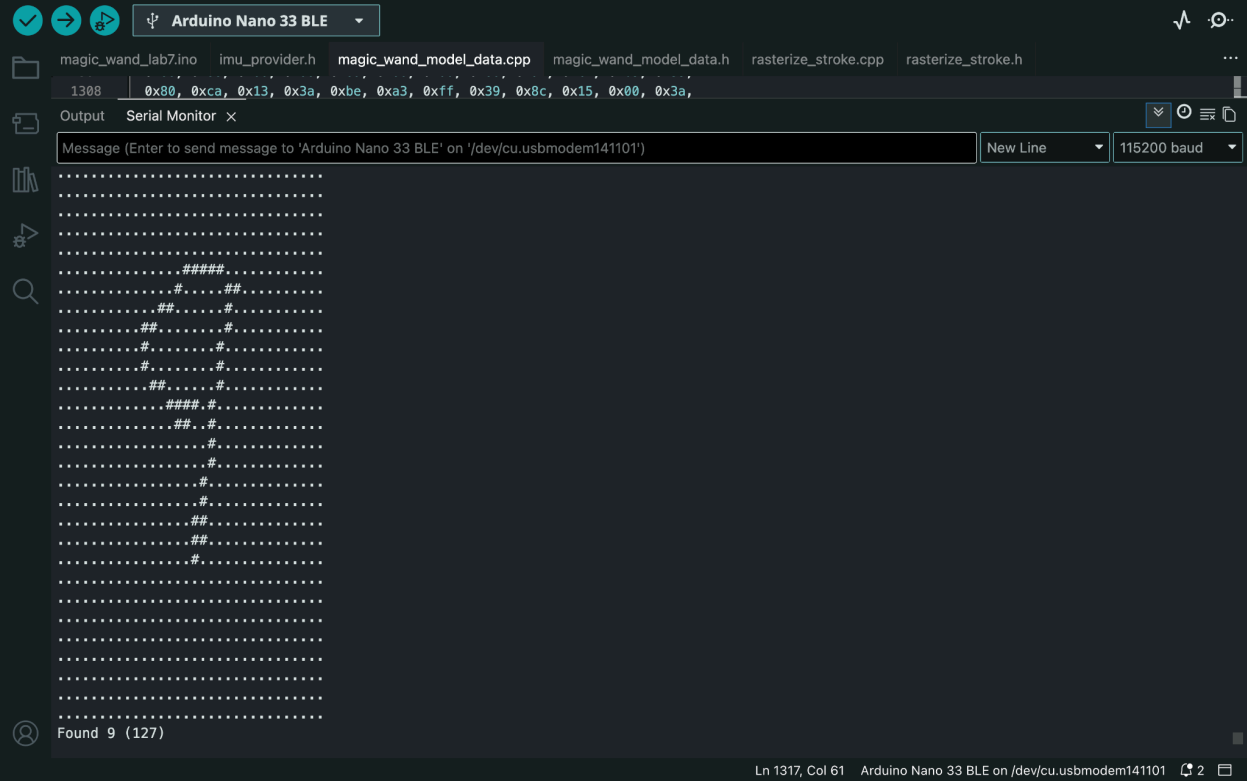
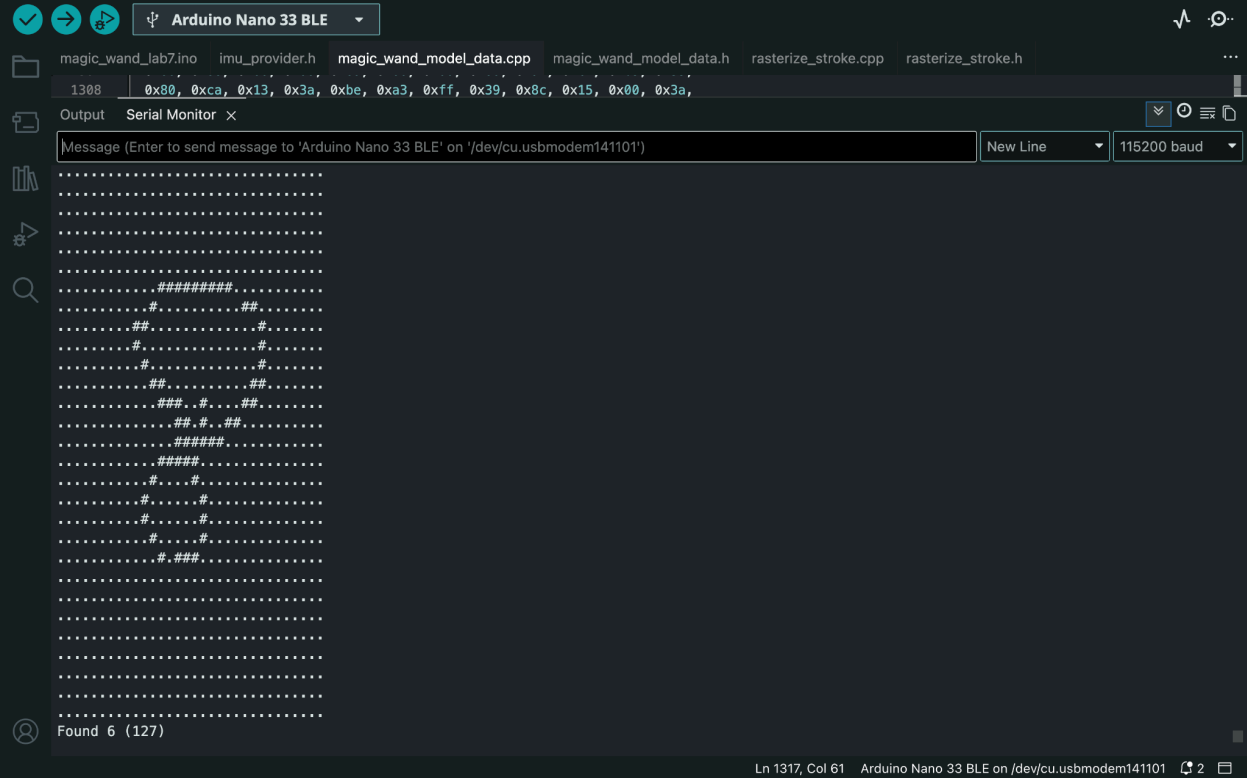
Baseline model

The screenshot shows the Arduino IDE interface with the 'Serial Monitor' window open. The top toolbar includes icons for checking, running, and uploading code, along with a dropdown menu set to 'Arduino Nano 33 BLE'. The file explorer on the left shows several files: 'magic_wand_lab7.ino', 'imu_provider.h', 'magic_wand_model_data.cpp', 'magic_wand_model_data.h', 'rasterize_stroke.cpp', and 'rasterize_stroke.h'. The main editor window displays the 'magic_wand_model_data.cpp' file, with line 1308 highlighted, containing a hexadecimal string: `0x80, 0xca, 0x13, 0x3a, 0xbe, 0xa3, 0xff, 0x39, 0x8c, 0x15, 0x00, 0x3a,`. The Serial Monitor window shows a message input field with the text 'Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem141101')', a 'New Line' button, and a baud rate dropdown set to '115200 baud'. The output area displays a series of dots and hash symbols, indicating the model's output. The status bar at the bottom shows 'indexing: 24/35' and 'Ln 1317, Col 61 Arduino Nano 33 BLE on /dev/cu.usbmodem141101'.

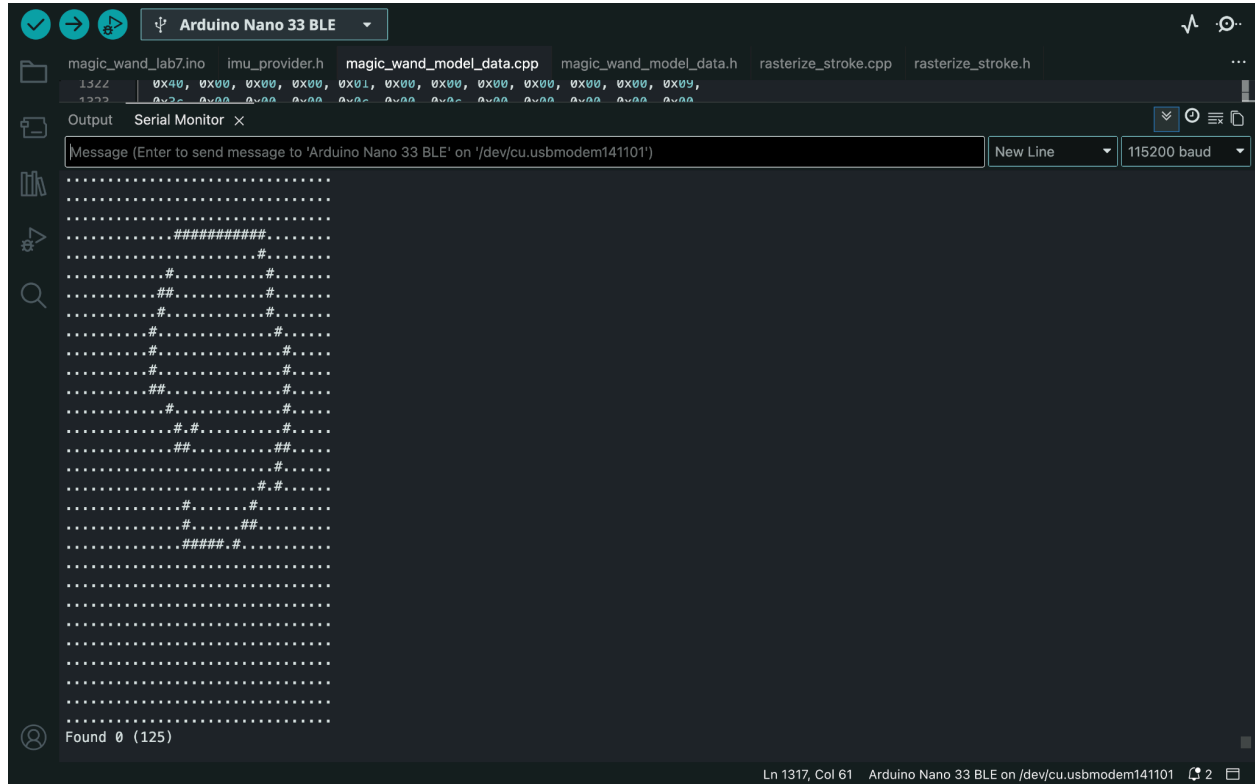
This screenshot is similar to the one above, showing the Arduino IDE interface. The file explorer and main editor window are the same. The Serial Monitor window shows the same message input field and baud rate. The output area displays a different series of dots and hash symbols, indicating a different model output. The status bar at the bottom shows 'indexing: 27/35' and 'Ln 1317, Col 61 Arduino Nano 33 BLE on /dev/cu.usbmodem141101'.

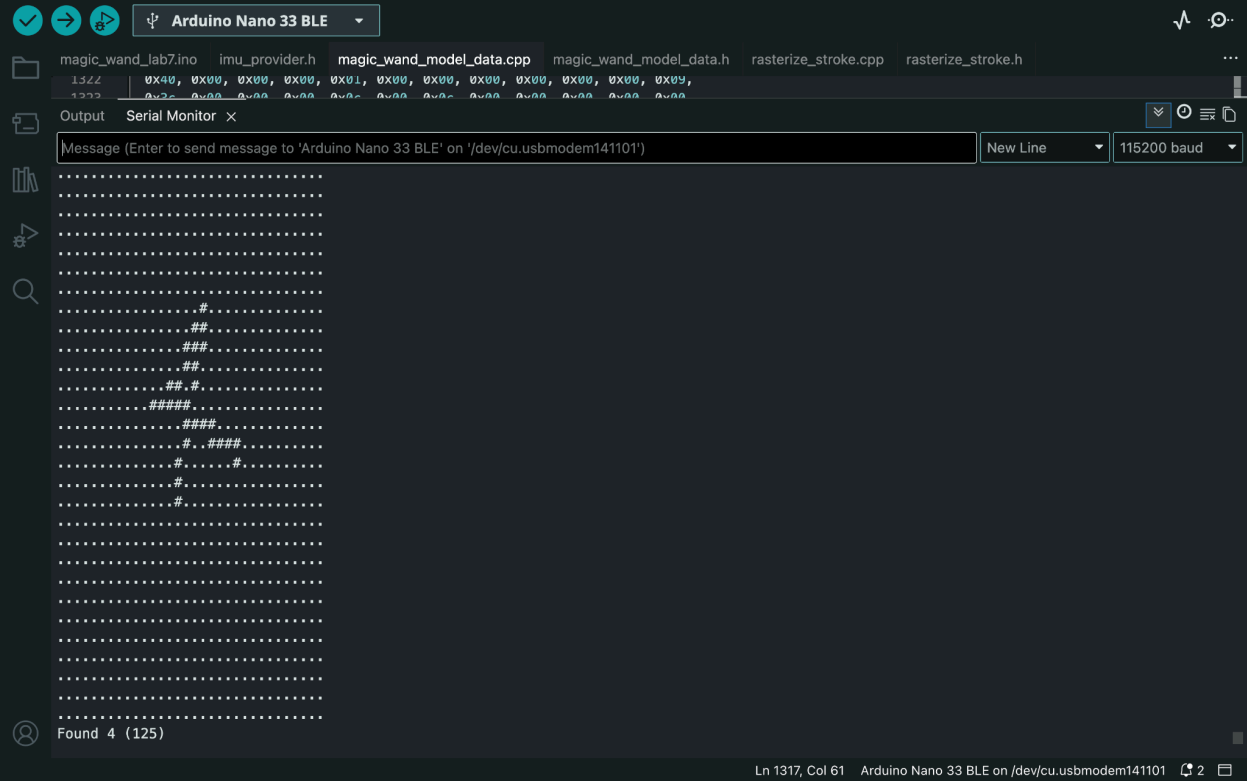
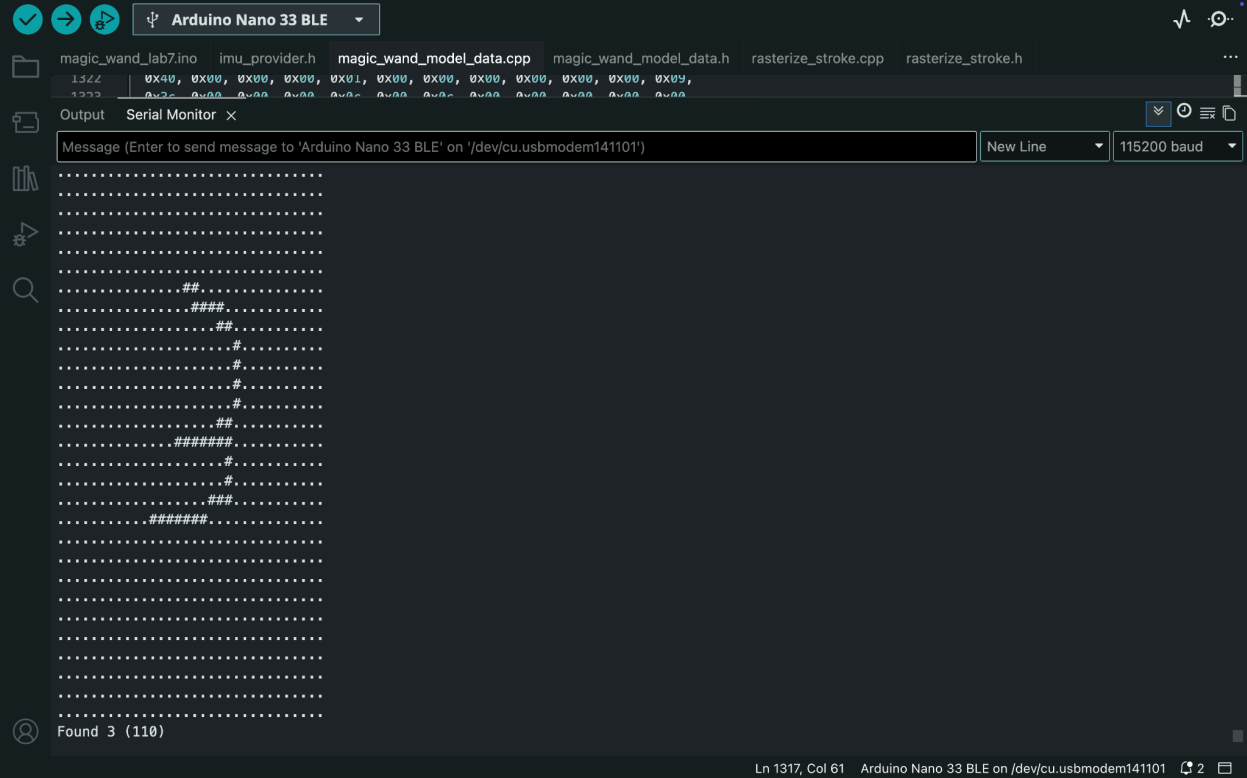


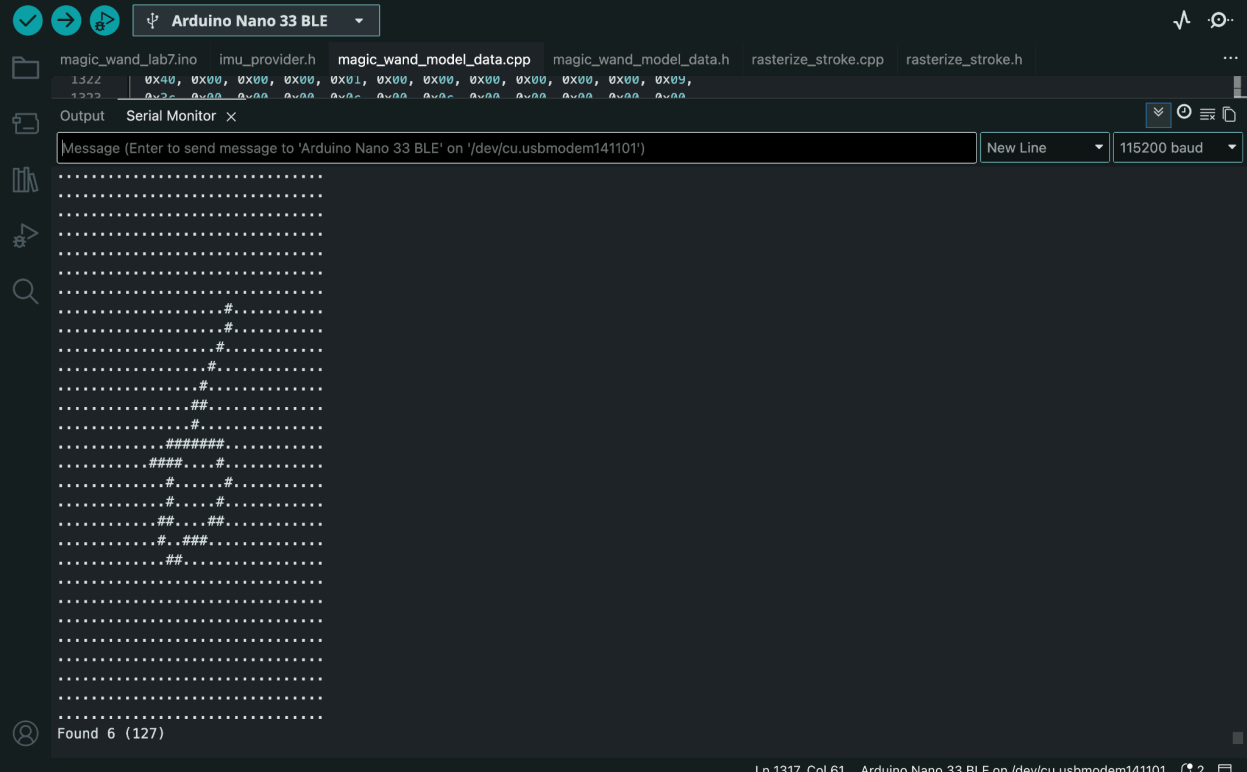
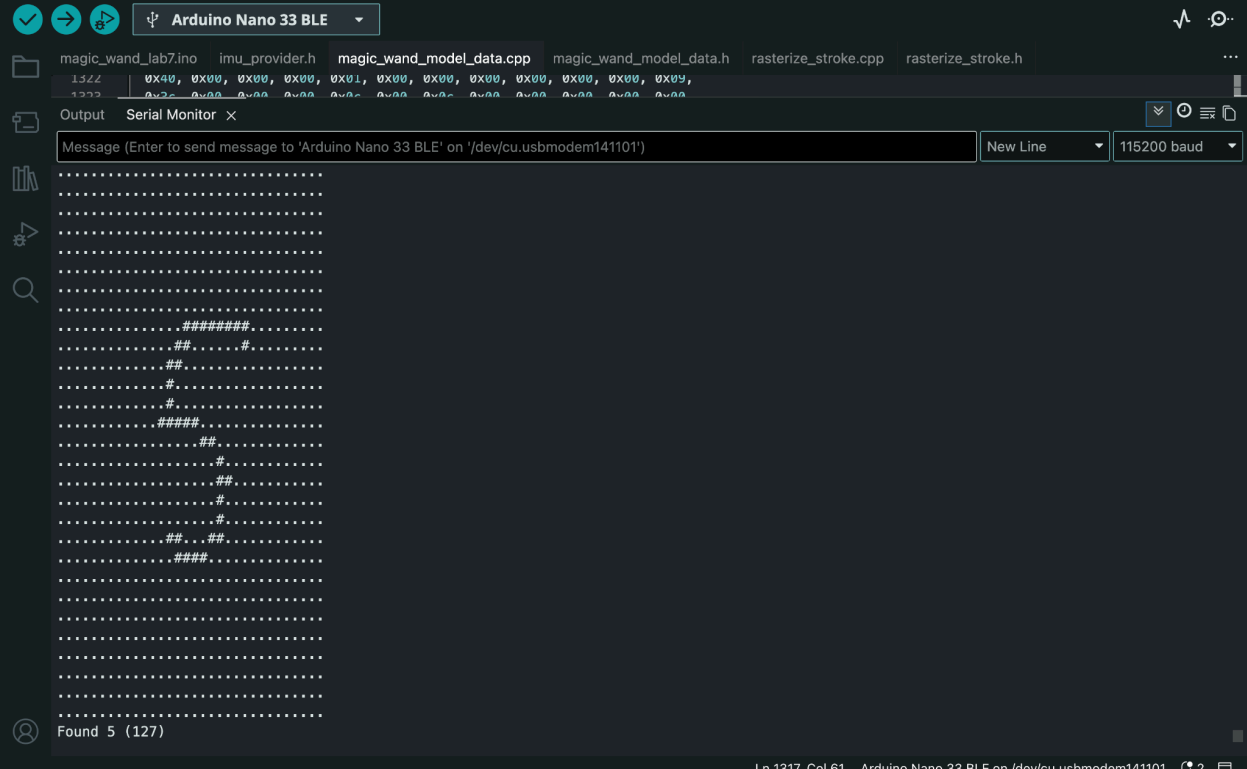




Finetuned model







The screenshot shows the Arduino IDE interface with the 'Serial Monitor' window open. The title bar indicates 'Arduino Nano 33 BLE'. The file explorer shows the project files: 'magic_wand_lab7.ino', 'imu_provider.h', 'magic_wand_model_data.cpp', 'magic_wand_model_data.h', 'rasterize_stroke.cpp', and 'rasterize_stroke.h'. The Serial Monitor window displays the output of the program, which is a series of dots forming a grid. Some dots are replaced by '#' characters, indicating the detected points. The Serial Monitor interface includes a message input field, a 'New Line' button, and a baud rate dropdown set to '115200 baud'.

[illegible]